

Computer Organization and Architecture

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Note: These notes have been prepared primarily from the following books:

- (1) **Computer System Architecture** by M. Morris Mano
- (2) **Computer Organization** by Carl Hamacher, Zvonko Vranesic, Safwat Zaky
- (3) **Operating System Concepts** by the Silberschatz, Galvin, Gagne

Please refer to the above books for more details.

Contents

1	Number Representation	9
1.1	Floating-Point Representation	9
1.2	Normalized Floating-Point Numbers	9
2	Register Transfer and Microoperations	10
2.1	Micro-operation	10
2.2	Register Transfer Language	10
2.3	Registers	10
2.3.1	Register Transfer	10
2.3.2	Control Function	11
2.4	Bus and Memory Transfers	11
2.4.1	Common Bus	11
2.4.2	Bus System for 4 Registers	11
2.5	Bus Transfer	11
2.6	Memory Transfer	11
2.6.1	Micro-operations	12
3	Arithmetic Microoperations	13
3.1	Basic Operations	13
3.2	Arithmetic Shift	13
4	Logic Microoperations	14
4.1	Basic Logic Operations	14
4.2	List of Logic Microoperations	14
5	Shift Microoperations	15
5.1	Shift-Left Operation	15
5.2	Shift-Right Operation	15
5.3	Logical Shift	15
5.4	Circular Shift	15
5.5	Arithmetic Shift	16
5.5.1	Checking for overflow	16
6	Arithmetic Logic Shift Unit (ALU)	17
6.1	Overview	17
6.2	Checking for overflow	17

7	Machine Instructions and Addressing Modes	18
7.1	Instruction Formats	18
7.2	Types of CPU Organizations	18
7.2.1	Single Accumulator Organization	19
7.2.2	General Register Organization	19
7.2.3	Stack Organization	19
8	CISC and RISC	20
8.1	CISC	20
8.2	RISC	20
9	Addressing Modes	22
9.1	Implied Mode	22
9.2	Immediate Mode	23
9.3	Register Mode	23
9.4	Register Indirect Mode	23
9.5	Autoincrement or Autodecrement Mode	23
9.6	Effective Address	23
9.7	Direct Address Mode	23
9.8	Indirect Address Mode	23
9.9	Relative Address Mode	24
9.10	Indexed Addressing Mode	24
9.11	Base Register Addressing Mode	24
10	Central Processing Unit	25
10.1	Overview	25
10.2	Some Fundamental Concepts	25
11	Single-Bus Organization of the Data Path Inside the CPU	26
11.1	Steps to Execute an Instruction	26
12	Execution of a Complete Instruction	28
12.1	Steps Required to Execute the Above Instruction	28
13	CPU Control Design	29
13.1	Hardwired Control Organization	29
13.2	Microprogrammed Control Organization	29
14	Micro-programmed Control	30
14.1	Overview	30
14.2	Control Word	30
14.3	Basic Organization of a Microprogrammed Control Unit	30
15	Microinstructions	32
15.1	Overview	32
15.2	Vertical Microinstructions	32
15.3	The Length of the Micro-instructions Can Be Reduced Chiefly Because	33
16	Prefetching Microinstructions	34
16.1	Overview	34

17 Instruction Pipeline	35
17.1 Overview	35
17.2 Instruction Cycle	35
18 Four-Segment Instruction Pipeline	36
19 Pipeline Conflicts	37
19.1 Handling of Branch Instructions or Control Hazard	37
19.1.1 Prefetch Target Instruction	37
19.1.2 Branch Prediction	37
19.1.3 Delayed Branch	37
20 Resolving Data Dependency Conflicts or Data Hazards	38
20.1 Hardware Interlocks	38
20.2 Operand Forwarding	38
20.3 Delayed Load	38
21 Hazards	39
22 Synchronous Pipelines	40
22.1 Asynchronous Pipelines	40
23 I/O Interface	41
23.1 The I/O Bus Consists Of	41
24 I/O versus Memory Bus	42
24.1 Overview	42
24.2 In Isolated I/O	42
24.3 In Memory Mapped I/O	42
25 Types of Interrupts	43
25.1 External Interrupts	43
25.2 Internal Interrupts	43
25.3 Software Interrupts	43
25.4 Interrupt-Request Line	43
26 Priority Interrupt	44
26.1 Overview	44
26.2 Establishing the Priority of Simultaneous Interrupts	44
26.2.1 Polling	44
26.2.2 Daisy-Chaining Priority	44
26.2.3 Parallel Priority Interrupt	44
26.3 Maskable Interrupt	45
26.4 Non-maskable Interrupt	45
26.5 Inter Processor Interrupt	45
26.6 Spurious Interrupt	45

27 Modes of Data Transfer	46
27.1 Programmed I/O	46
27.2 Interrupt-Initiated I/O	46
27.3 Direct Memory Access (DMA)	47
28 Input-Output Processor (IOP)	48
28.1 Overview	48
29 Direct Memory Access	49
29.1 Overview	49
29.1.1 Bus Request	49
29.1.2 Bus Grant	49
29.1.3 Burst Transfer	49
29.1.4 Cycle Stealing	49
30 Blocking and Nonblocking I/O	50
30.1 Blocking I/O	50
31 Asynchronous I/O	51
32 Buffering	52
32.1 Overview	52
32.2 Spooling	52
32.3 Expansion Bus and PCI Bus	52
33 Polling	53
34 Memory Management	54
34.1 Raster Display	54
34.2 Memory Hierarchy in a Computer System	54
34.3 Protection of Memory Space	54
35 Static RAM	56
35.1 Overview	56
36 Dynamic RAM	57
36.1 Overview	57
37 RAM and ROM Chips	58
37.1 RAM Chip	58
37.1.1 Bi-directional Bus	58
37.2 ROM Chip	58
37.2.1 Uni-directional Bus	58
37.3 Memory Address Map	58
38 ROM	59
38.1 Bootstrap Loader	59

39 Associative Memory	60
39.1 Overview	60
39.2 Block Diagram of Associative Memory	60
40 Hit Ratio	61
40.1 Overview	61
41 Cache Memory	62
41.1 Locality of Reference	62
41.2 Mapping	62
42 Direct Mapping	63
42.1 Overview	63
43 Set-Associative Mapping	64
43.1 Overview	64
44 Writing into Cache	65
44.1 Overview	65
44.2 Cache Initialization	65
44.3 Cache Coherence	65
45 Memory Binding	66
45.1 Overview	66
45.2 Dynamic Loading	66
46 Dynamic Linking	67
46.1 Overview	67
47 Virtual Memory	68
48 Logical Versus Physical Address Space	69
48.1 Overview	69
49 Contiguous Memory Allocation	70
49.1 Overview	70
49.2 External Fragmentation	70
50 Paging	71
50.1 Overview	71
50.2 Mapping from Address Space to Memory Space	71
50.2.1 Page (p) Line Number (d)	71
51 Translation Look-aside Buffer (TLB)	72
52 Valid-Invalid Bit	73
52.1 Demand Paging	73

53 Page Replacement Algorithms	74
53.1 FIFO Page Replacement	74
53.1.1 Belady's Anomaly	74
53.2 Optimal Page Replacement	74
53.3 LRU Page Replacement	74
54 Thrashing	75
55 Local vs. Global Page Replacement	76
56 Byte Addressing vs. Word Addressing	77
56.1 Byte Addressing	77
56.2 Word Addressing	77

Chapter 1

Number Representation

1.1 Floating-Point Representation

The floating-point representation of a number has two parts:

- (i) **Mantissa:** Signed fixed-point number
- (ii) **Exponent:** Designates the position of the decimal (or binary) point

For example, the number +6132.789 is represented in floating-point as:

$$\begin{aligned}\text{Fraction} &= +0.6132789 \\ \text{Exponent} &= +04\end{aligned}$$

The value of the exponent indicates that the actual position of the decimal point is four positions to the right of the indicated decimal point in the fraction.

Another example: The binary number +1001.11 is represented with an 8-bit fraction and 6-bit exponent as follows:

$$\begin{aligned}\text{Fraction} &: +01001110 \\ \text{Exponent} &: 000100\end{aligned}$$

The fraction has a 0 in the leftmost position to denote positive. The binary point of the fraction follows the sign bit, but is not shown in the register. The exponent has the equivalent binary number 4. The actual binary point in the fraction is four positions to the right of the indicated binary point in the fraction.

1.2 Normalized Floating-Point Numbers

A floating-point number is said to be normalized if the most significant digit of the mantissa is nonzero.

For example, the decimal number 350 is normalized, but 00035 is not.

Normalized numbers provide the maximum possible precision for the floating-point number.

A zero cannot be normalized because it does not have a nonzero digit (if a nonzero digit d is usually represented in floating-point by all 0s in the mantissa and exponent).

Chapter 2

Register Transfer and Microoperations

2.1 Micro-operation

The operations executed on data stored in registers are called microoperations. A micro-operation is an elementary operation such as shift, count, clear, and load.

2.2 Register Transfer Language

The symbolic notation used to describe the microoperation transfers among registers is called a Register Transfer Language (RTL).

The internal hardware organization of a digital computer is best defined by specifying:

- (i) The set of registers it contains and their function
- (ii) The sequence of microoperations performed on the binary information stored in the registers
- (iii) The control that initiates the sequence of microoperations

2.3 Registers

- (i) **Memory Address Register (MAR):** Holds an address of the memory unit
- (ii) **PC (Program Counter):**
- (iii) **IR (Instruction Register):**
- (iv) **R_i (Processor Register):**

Computer registers are designated by capital letters.

2.3.1 Register Transfer

$$R_2 \leftarrow R_1$$

2.3.2 Control Function

$$P : R_2 \leftarrow R_1$$

P represents the transfer operation to be executed by the hardware only if $P = 1$.

Note: The clock is not included as a variable in the register transfer statements. It is assumed that all transfers occur during a clock edge transition.

2.4 Bus and Memory Transfers

2.4.1 Common Bus

A common bus consists of a set of common lines, one for each bit of a register, through which binary information is transferred one at a time.

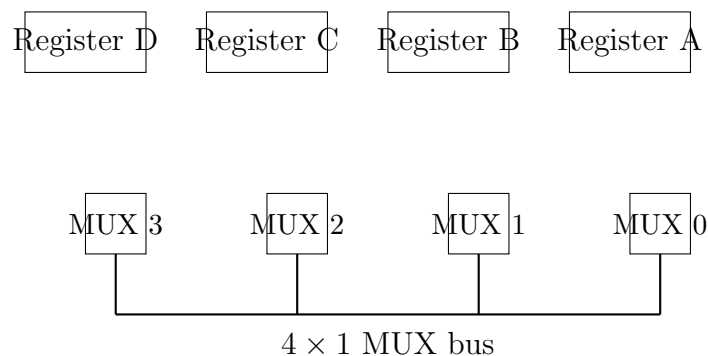
Control signals determine which register is selected by the bus during each particular register transfer.

2.4.2 Bus System for 4 Registers

In general, a bus system will multiplex k registers of n bits each to produce an n -line common bus.

The number of multiplexers needed to construct the bus is equal to n , the number of bits in each register.

The size of each multiplexer must be $k \times 1$, since it multiplexes k data lines.



2.5 Bus Transfer

$$\begin{aligned} \text{BUS} &\leftarrow R_2, & R_1 &\leftarrow \text{BUS} \\ \Rightarrow R_1 &\leftarrow R_2 \end{aligned}$$

2.6 Memory Transfer

Read: $\text{DR} \leftarrow M[\text{AR}]$

Write: $M[\text{AR}] \leftarrow R_1$

where DR is the Data Register and AR is the Address Register.

2.6.1 Micro-operations

The microoperations are classified into four categories:

- (i) Register Transfer Microoperations
- (ii) Arithmetic Microoperations
- (iii) Logic Microoperations
- (iv) Shift Microoperations

Chapter 3

Arithmetic Microoperations

3.1 Basic Operations

- (i) **Addition:** $R_3 \leftarrow R_1 + R_2$
- (ii) **Subtraction:** $R_3 \leftarrow R_1 - R_2$ or $R_3 \leftarrow R_1 + \overline{R_2} + 1$
- (iii) **1's complement:** $R_2 \leftarrow \overline{R_2}$
- (iv) **2's complement:** $R_2 \leftarrow \overline{R_2} + 1$
- (v) **Increment:** $R_1 \leftarrow R_1 + 1$
- (vi) **Decrement:** $R_1 \leftarrow R_1 - 1$

3.2 Arithmetic Shift

Explained later with shift microoperations.

The increment and decrement microoperations are implemented with a binary up-down counter.

Chapter 4

Logic Microoperations

These operations consider each bit of the register separately and treat them as binary variables.

4.1 Basic Logic Operations

- (i) **OR:** $R_1 \leftarrow R_2 \vee R_3$
- (ii) **AND:** $R_1 \leftarrow R_2 \wedge R_3$
- (iii) **Ex-OR:** $R_1 \leftarrow R_2 \oplus R_3$
- (iv) **Complement:** $R_1 \leftarrow \overline{R_1}$

The symbol $+$ occurs in a micro-operation, it will denote an arithmetic plus. When it occurs in a control function, it will denote an OR operation.

For example, in the statement:

$$P + Q : \quad R_1 \leftarrow R_2 + R_3, \quad R_4 \leftarrow R_5 \vee R_6$$

the OR operation is implied and the addition is implied in the microoperation.

4.2 List of Logic Microoperations

There are 16 different logic operations that can be performed with two binary variables:

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7
0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1

x	y	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1

Chapter 5

Shift Microoperations

There are three types of shifts:

- (i) Logical Shift
- (ii) Circular Shift
- (iii) Arithmetic Shift

5.1 Shift-Left Operation

The serial input transfers a bit into the rightmost position.

5.2 Shift-Right Operation

The serial input transfers a bit into the leftmost position.

5.3 Logical Shift

A logical shift is one that transfers 0 through the serial input. If only all zeros and symbols are used for logical left shift and right shift microoperations:

$$\begin{aligned}R_1 &\leftarrow \text{shl } R_1 \\R_2 &\leftarrow \text{shr } R_2\end{aligned}$$

5.4 Circular Shift

Here, the serial output of the shift register is connected to its serial input.

Examples: c_i and c_o symbols are used for circular shift left and right microoperations:

$$\begin{aligned}R_1 &\leftarrow \text{cir } R_1 \\R_2 &\leftarrow \text{cil } R_2\end{aligned}$$

5.5 Arithmetic Shift

An arithmetic shift-left (ashl) multiplies a signed binary number by 2.

An arithmetic shift-right (ashr) divides the number by 2.

Arithmetic shifts must leave the sign bit unchanged because the sign of the number remains the same when it is multiplied or divided by 2.

The ashr leaves the sign bit unchanged and shifts the number (including the sign bit) to the right.

The ashl inserts a 0 into R_0 from right and shifts all other bits to the left. If the multiplication by 2 causes an overflow, the sign-bit in the register changes from 0 to 1 or 1 to 0.

5.5.1 Checking for overflow

If $V_s = 0$: There is no overflow.

If $V_s = 1$: There is an overflow.

Where $V_s = R_{n+1} \oplus R_{n-2}$.

Chapter 6

Arithmetic Logic Shift Unit (ALU)

6.1 Overview

To perform a microoperation, in ALU, the contents of specified registers are placed in the inputs of the common ALU. The ALU performs an operation and the result of the operation is then transferred to a destination register.

The ALU is a combinational circuit so that the entire operation can be performed during one clock pulse period.

6.2 Checking for overflow

$$\begin{array}{lll} \text{ashr} \rightarrow |R_{n+1}|R_{n+2} & \longrightarrow & |R_1|R_0| \\ \text{ashl} \rightarrow |R_{n+1}|R_{n+2}| & \longleftarrow & |R_1|R_0| \times 2 \end{array}$$

If $V_s = 0$: There is no overflow.

If $V_s = 1$: There is an overflow.

Chapter 7

Machine Instructions and Addressing Modes

7.1 Instruction Formats

A computer will usually have a variety of instruction code formats. It is the function of the control unit within the CPU to interpret each instruction code and provide the necessary control functions needed to process the instruction.

The bits of the instruction code are usually divided into three groups:

- (i) An operation code field that specifies the operation to be performed
- (ii) An address field that designates a memory address or a processor register
- (iii) A mode field that specifies the way the operand or the effective address is determined

Operation Code	Address Field	Mode
----------------	---------------	------

A register address is a binary number of k bits that define one of 2^k registers in the CPU.

The instruction code is also called a control word.

An operation code is sometimes called a macrooperation because it specifies a set of micro-operations.

7.2 Types of CPU Organizations

The number of address fields in the instruction format of a computer depends on the internal organization of its registers.

Most computers fall into one of three types of CPU organizations:

- (i) Single Accumulator organization
- (ii) General Register Organization
- (iii) Stack Organization

7.2.1 Single Accumulator Organization

All operations are performed with an implied accumulator register. The instruction format in this type of computer uses one address field.

For example:

$$\text{ADD } X \Rightarrow AC \leftarrow AC + M[X]$$

where AC is the Accumulator Register.

7.2.2 General Register Organization

The instruction format in this type of computer uses either three or two address fields.

Examples:

- (i) $\text{ADD } R_1, R_2, R_3 \Rightarrow R_1 \leftarrow R_2 + R_3$
- (ii) $\text{ADD } R_1, R_2 \Rightarrow R_1 \leftarrow R_1 + R_2$
- (iii) $\text{ADD } R_1, X \Rightarrow R_1 \leftarrow R_1 + M[X]$
- (iv) $\text{MOV } R_1, R_2 \Rightarrow R_1 \leftarrow R_2$

7.2.3 Stack Organization

Computers with stack organization have PUSH and POP instructions which require one address field.

For example: PUSH X

Operation-type instructions do not need an address field (zero address instruction).

For example: ADD

Single Accumulator Organization uses one-address instructions.

General Register Organization uses three- and two-address instructions.

Stack Organization uses zero- and one-address instructions.

In a general register's CPU organization, there are eight general purpose registers and ALU can perform 32 different operations.

The number of selection lines for selecting the operand is: $3 = (2^3 = 8)$

The number of bits in operation code = 5 ($2^5 = 32$)

The length of the control word = $5 + 3 + 3 + 3 = 14$ (operation + instruction)

Chapter 8

CISC and RISC

8.1 CISC

A computer with a large number of instructions is classified as a complex instruction set computer (CISC).

The central goal of a CISC architecture is to attempt to provide a single machine instruction for each statement that is written in a high-level language.

The major characteristics of CISC architecture are:

- (1) A large number of instructions - typically from 100 to 250 instructions
- (2) Some instructions that perform specialized tasks and are used infrequently
- (3) A large variety of addressing modes - typically from 5 to 20 different modes
- (4) Variable-length instruction formats
- (5) Instructions that manipulate operands in memory

8.2 RISC

The Reduced Instruction Set Computer is a computer containing fewer instruction with simple constructs that can be executed much faster within the CPU without having to use memory as often.

The major characteristics of a RISC processor are:

- (1) Relatively few instructions
- (2) Relatively few addressing modes
- (3) Memory access limited to load and store instructions
- (4) All operations done within the registers of the CPU
- (5) Fixed-length, easily decoded instruction format
- (6) Single-cycle instruction execution
- (7) Hardwired rather than microprogrammed control

- (8) Delayed Load: When there is a data dependency conflict in instruction pipeline
- (9) Delayed Branch: When there is a branch instruction in instruction pipeline

Chapter 9

Addressing Modes

The way the operands are chosen during program execution is dependent on the addressing mode of the instruction.

The addressing mode may reduce the number of bits in the addressing field of the instruction.

Different types of addressing modes are given below:

- (1) Implied Mode
- (2) Immediate Mode
- (3) Register Mode
- (4) Register Indirect Mode
- (5) Autoincrement or Autodecrement Mode
- (6) Direct Address Mode
- (7) Indirect Address Mode
- (8) Relative Address Mode
- (9) Indexed Addressing Mode
- (10) Base Register Addressing Mode

9.1 Implied Mode

In this mode, the operands are specified implicitly in the definition of the instruction.

For example, all register reference instructions that use only an accumulator are implied-mode instructions.

Zero-address instructions in a stack-organized computer are implied-mode instructions.

9.2 Immediate Mode

In this mode, the operand is specified in the instruction itself.

In other words, an immediate-mode instruction has an operand field rather than an address field.

For example: $\text{LD } \#NBR \Rightarrow AC \leftarrow NBR$

9.3 Register Mode

In this mode, the operands are in registers that reside within the CPU. The particular register is selected from a register field in the instruction.

A k -bit field can specify any one of 2^k registers.

Example: $\text{LD } R_1 \Rightarrow AC \leftarrow R_1$

9.4 Register Indirect Mode

In this mode, the instruction specifies a register that reside within the CPU whose contents give the address of the operand in memory.

For example: $\text{LD } (R_1) \Rightarrow AC \leftarrow M[R_1]$

9.5 Autoincrement or Autodecrement Mode

This is similar to the register indirect mode except that the register is incremented or decremented after (or before) its value is used to access the memory.

For example:

$$\text{LD } (R_1)+ \Rightarrow AC \leftarrow M[R_1], \quad R_1 \leftarrow R_1 + 1$$

9.6 Effective Address

The effective address is defined to be the memory address obtained from the computation dictated by the given addressing mode.

9.7 Direct Address Mode

In this mode, the effective address is equal to the address part of the instruction.

For example: $\text{LD } ADR \Rightarrow AC \leftarrow M[ADR]$

9.8 Indirect Address Mode

In this mode, the address field of the instruction gives the address where the effective address is stored in memory.

For example: $\text{LD } @ADR \Rightarrow AC \leftarrow M[M[ADR]]$

9.9 Relative Address Mode

In this mode, the content of the program counter (PC) is added to the address part of the instruction in order to obtain the effective address.

For example: $\text{LD } \$\text{ADR} \Rightarrow \text{AC} \leftarrow M[\text{PC} + \text{ADR}]$

Relative addressing is often used with branch-type instructions.

9.10 Indexed Addressing Mode

In this mode, the content of an index register is added to the address part of the instruction to obtain the effective address.

For example: $\text{LD } \text{ADR}(x) \Rightarrow \text{AC} \leftarrow M[\text{ADR} + x]$

9.11 Base Register Addressing Mode

In this mode, the content of a base register is added to the address part of the instruction to obtain the effective address.

For example:

The base register addressing mode is used for relocation of programs in memory.

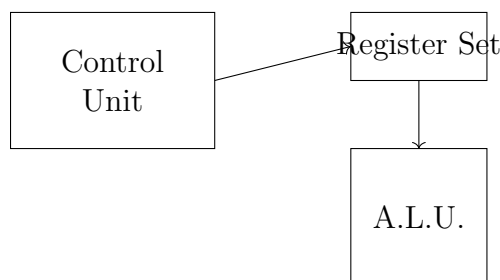
Chapter 10

Central Processing Unit

10.1 Overview

The CPU is made up of three major parts:

- (i) Register Set
- (ii) Arithmetic-Logic Unit (ALU)
- (iii) Control Unit (CU)



10.2 Some Fundamental Concepts

The instructions constituting a program to be executed are loaded in sequential locations in the main memory.

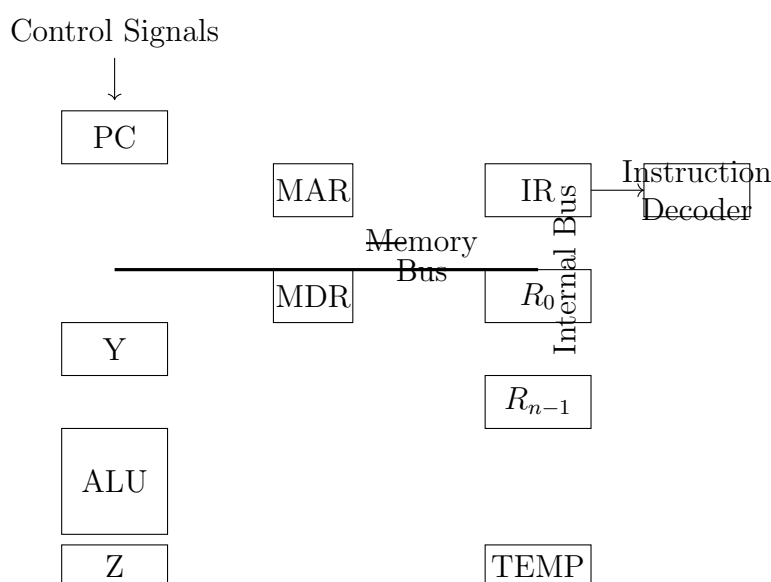
Instructions are fetched from successive memory locations until a branch or a jump instruction is executed.

A dedicated CPU register, referred to as the program counter (PC), contains the address of the memory location containing the next instruction.

After fetching an instruction, the contents of the PC are updated to point to the next instruction in sequence.

Chapter 11

Single-Bus Organization of the Data Path Inside the CPU



Here, the single common bus is internal to the CPU and should not be confused with the external bus.

The ALU and the registers are used for storing and manipulating data and referred to as a datapath.

11.1 Steps to Execute an Instruction

- (i) Fetch the contents of the memory location pointed to by PC (program counter) and load them into IR (instruction register):

$$IR \leftarrow M[PC]$$

- (ii) Increment the contents of PC by 1:

$$PC \leftarrow PC + 1$$

- (iii) Carry out the actions specified by the instruction in the IR

Steps 1 and 2 are usually referred to as the fetch phase.

Step 3 constitutes the execution phase.

In cases where an instruction occupies more than one word, steps 1 and 2 must be repeated as many times as necessary to fetch the complete instruction.

The width of the data bus between the CPU and the main memory determines the number of bytes that can be transferred in one access. This width is likely to be $\frac{1}{2}$, 4, or 8 bytes, so the PC must be incremented by 2, 4, or 8 after each fetch step.

Chapter 12

Execution of a Complete Instruction

Consider the instruction:

$$\text{ADD } (R_3), R_1 \Rightarrow R_1 \leftarrow M[R_3] + R_1$$

12.1 Steps Required to Execute the Above Instruction

Step	Action
1	PC _{out} , MAR _{in} , Read, Clear Y, Set Carry-in to ALU ADD _{in}
2	Z _{out} , PC _{in} , WMFC
3	MDR _{out} , IR _{in}
4	R3 _{out} , MAR _{in} , Read
5	R1 _{out} , Y _{in} , WMFC
6	MDR _{out} , Add, Z _{in}
7	Z _{out} , R1 _{in} , End.

WMFC $\rightarrow$ Wait for Memory Function Completed.

For above instruction execution, 7 non-overlapping time-slots are required. Each time slot must be at least long enough for the functions specified in the corresponding to be completed.

Chapter 13

CPU Control Design

There are two major types of control organization:

- (i) Hardwired Control
- (ii) Microprogrammed Control

13.1 Hardwired Control Organization

In the hardwired organization, the control logic is implemented with gates, flip-flops, decoders, and other digital circuits.

It has the advantage that it can be optimized to produce a fast mode of operation.

It requires changes in the wiring among the various components if the design has to be modified or changed.

13.2 Microprogrammed Control Organization

In the micro-programmed organization, the control information is stored in a control memory. The control memory is programmed to initiate the required sequence of micro-operations.

In the micro-programmed control, any required changes or modifications can be done by updating the microprogram in control memory.

Chapter 14

Micro-programmed Control

14.1 Overview

In micro-programmed control, the control signals are generated by a program similar to the machine language programs.

14.2 Control Word

A control word (CW) is a word whose individual bits represent the various control signals.

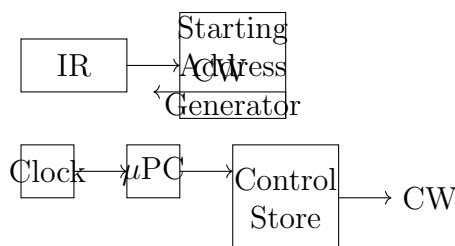
A sequence of CWs corresponding to the control sequence of a machine instruction constitutes the micro-routine.

The individual CWs in this micro-routine are referred to as micro-instructions.

The microroutines for all instructions in the instruction set of a computer are stored in a special memory called the control store.

The control unit can generate the control signals for any instruction by sequentially reading the CWs of the corresponding microinstruction from the control store.

14.3 Basic Organization of a Microprogrammed Control Unit



The microprogram counter (μ PC) is used to read the control words sequentially from the control store.

Every time a new instruction is loaded into the IR, the output of the block labeled starting address generator is loaded into the μ PC. The μ PC is then automatically incremented by the clock, causing successive microinstructions to be read from the control store.

The μ PC does not increment in the following situations:

- (i) When an End microinstruction is encountered
- (ii) When a new instruction is loaded into IR
- (iii) When a branch micro-instruction is encountered and the branch condition is satisfied

Chapter 15

Microinstructions

15.1 Overview

A straightforward way to structure microinstructions is to assign one bit position to each control signal required in the CPU.

Control Signals F0-F7:

Micro-instruction	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7
1	0	1	1	1	0	0	0	1
2	1	0	0	0	0	0	0	0
3	0	0	0	0	1	1	0	0
4	0	0	1	1	0	0	0	0
5	0	0	0	0	0	0	1	0
6	0	0	0	0	1	0	0	0
7	0	0	0	0	0	0	0	0

Control Signals F8-F15:

Micro-instruction	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
1	1	1	1	0	0	0	0	0
2	0	0	0	1	0	0	0	1
3	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	1	0
5	0	0	0	0	1	0	0	0
6	0	1	1	0	0	0	0	0
7	0	1	0	1	0	0	1	0

This scheme has one serious drawback: assigning individual bits to each control signal results in long microinstructions.

The above scheme is called a horizontal organization.

15.2 Vertical Microinstructions

Highly encoded schemes that use compact codes to specify only a small number of control functions in each microinstruction are referred to as a vertical organization.

Grouping control signals into fields requires a little more hardware because decoding circuits must be used to decode the bit patterns of each field into individual control signals.

15.3 The Length of the Micro-instructions Can Be Reduced Chiefly Because

- (i) Most signals are not needed simultaneously
- (ii) Many signals are mutually exclusive

Chapter 16

Prefetching Microinstructions

16.1 Overview

The major drawback of microprogrammed control is that it is slow.

Fetching a microinstruction from the control store takes considerably longer than generating control signals using hardwired circuits.

The performance of the microprogrammed control can be improved by prefetching the microinstruction while the current one is being executed.

Prefetching microinstructions presents some organizational difficulties because sometimes the status flags and the results of the currently executed microinstructions are needed to determine the address of the next microinstruction.

Chapter 17

Instruction Pipeline

17.1 Overview

An instruction pipeline reads consecutive instructions from memory while previous instructions are being executed in other segments.

This causes the instruction fetch and execute phases to overlap and perform simultaneous operations.

If an instruction causes a branch out of sequence, then the pipeline must be emptied and all the instructions that have been in memory after the branch instruction must be discarded.

17.2 Instruction Cycle

Generally, the computer needs to process each instruction with the following set sequence of steps:

- (1) Fetch the instruction from memory
- (2) Decode the instruction
- (3) Calculate the effective address
- (4) Fetch the operands from memory
- (5) Execute the instruction
- (6) Store the results in the proper place

Chapter 18

Four-Segment Instruction Pipeline

Suppose the instruction cycle is divided into four segments as given below:

- (i) FI is the segment that fetches an instruction
- (ii) DA is the segment that decodes the instruction and calculates the effective address
- (iii) FO is the segment that fetches the operand
- (iv) EX is the segment that executes the instruction

It is assumed that the processor has separate instruction and data memories so that the operation in FI and FO can proceed at the same time.

Step →	1	2	3	4	5	6	7	8	9	10	11	12
Instruction: 1	FI	DA	FO	EX								
2		FI	DA	FO	EX							
(Branch) 3			FI	DA	FO	EX						
4				FI	-	-	FI	DA	FO	EX		
5					-	-	-	FI	DA	FO	EX	
6									FI	DA	FO	EX
7										FI	DA	FO

Assume that instruction 3 is a branch instruction. As soon as this instruction is decoded in segment DA in step 4, the transfer from FI to DA of the other instructions is halted until the branch instruction is executed in step 6.

If the branch is taken, a new instruction is fetched in step 7.

If the branch is not taken, the instruction fetched previously in step 4 can be used.

Chapter 19

Pipeline Conflicts

In general, there are three major difficulties that cause the instruction pipeline to deviate from its normal operation:

- (i) **Resource Conflicts:** Caused by access to memory by two segments at the same time
- (ii) **Data dependency conflicts:** Arise when an instruction depends on the result of a previous instruction, but this result is not yet available
- (iii) **Branch difficulties:** Arise from branch and other instructions that change the value of PC

19.1 Handling of Branch Instructions or Control Hazard

19.1.1 Prefetch Target Instruction

Prefetch the target instruction in addition to the instruction following the branch. Both are saved until the branch is executed.

19.1.2 Branch Prediction

Some additional logic is used to guess the outcome of a conditional branch instruction before it is executed. The pipeline then begins prefetching the instruction stream from the predicted path.

19.1.3 Delayed Branch

Used in most RISC processors. In this procedure, the compiler detects the branch instructions and rearranges the machine language code sequence by inserting useful instructions that keep the pipeline operating without interruptions.

Chapter 20

Resolving Data Dependency Conflicts or Data Hazards

20.1 Hardware Interlocks

An interlock is a circuit that detects instructions whose source operands are destinations of instructions farther up in the pipeline. After detecting, we delay the instruction by enough clock cycles to resolve the conflict.

20.2 Operand Forwarding

Avoid the conflict by routing the data through special paths between pipeline segments.

20.3 Delayed Load

Delay the loading of the conflicting data by inserting no-operation instructions (in compiler).

Chapter 21

Hazards

There are situations, called hazards, that prevent the next instruction in the instruction stream from being executing during its designated clock cycle. Hazards reduce the performance from the ideal speedup gained by the pipeline.

There are three classes of hazards:

- (i) **Structural Hazards:** They arise from resource conflicts when the hardware can not support all possible combinations of instructions in simultaneous overlapped execution
- (ii) **Data Hazards:** They arise when an instruction depends on the result of a previous instruction in a way that is exposed by the overlapping of instructions in the pipeline
- (iii) **Control Hazards:** They arise from the pipelining of branches and other instructions that change the PC

Hazards in pipeline can make it necessary to stall the pipeline.

Chapter 22

Synchronous Pipelines

These are clocked synchronously. The time between each clock signal is set to be greater than the longest delay between pipeline stages.

22.1 Asynchronous Pipelines

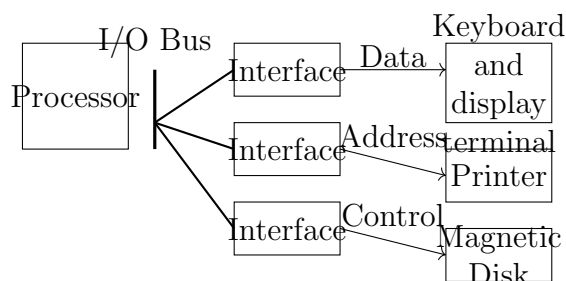
They use request/acknowledge system, wherein each stage can detect when it's finished.

Chapter 23

I/O Interface

I/O interfaces connect the peripherals to the CPU.

I/O interfaces supervise and synchronize all input and output transfers.



23.1 The I/O Bus Consists Of

- (i) Data Lines
- (ii) Address Lines
- (iii) Control Lines

Each interface decodes the address and control received from the I/O bus, interprets them for the peripheral, and provides signals for the peripheral controller.

Each peripheral has its own controller that operates the particular electromechanical device.

Address lines are used to select a peripheral.

Control lines are used to send commands to an interface.

Chapter 24

I/O versus Memory Bus

24.1 Overview

The memory bus contains data, address, and read/write control lines.

There are three ways that computer buses can be used to communicate with memory and I/O:

- (i) Use two separate buses, one for memory and the other for I/O (Using IOP - Input Output Processor)
- (ii) Use one common bus for both memory and I/O, but have separate control lines for each (Isolated I/O)
- (iii) Use one common bus for memory and I/O with common control lines (memory mapped)

24.2 In Isolated I/O

The I/O read and I/O write control lines are enabled during an I/O transfer. The memory read and memory write control lines are enabled during a memory transfer.

24.3 In Memory Mapped I/O

A segment of the total address space is reserved for interface registers.

Chapter 25

Types of Interrupts

There are three major types of interrupts that cause a break in the normal program execution:

- (i) External Interrupts
- (ii) Internal Interrupts
- (iii) Software Interrupts

25.1 External Interrupts

External interrupts come from external sources like I/O devices, timing devices, etc.

25.2 Internal Interrupts

Internal interrupts arise from illegal or erroneous use of an instruction or data. Internal interrupts are also called traps. Examples are: register overflow, attempt to divide by zero, etc.

25.3 Software Interrupts

Software interrupt is a special call instruction that behaves like an interrupt rather than a subroutine call.

25.4 Interrupt-Request Line

The CPU has a wire called the interrupt request line that the CPU senses after executing every instruction.

Chapter 26

Priority Interrupt

26.1 Overview

A priority interrupt is a system that establishes a priority over the various sources to determine which condition is to be served first when two or more requests arrive simultaneously.

The system may also determine which conditions are permitted to interrupt the computer while another interrupt is being serviced.

26.2 Establishing the Priority of Simultaneous Interrupts

It can be done by software or hardware. There are three methods to establish priority:

- (i) Polling
- (ii) Daisy-Chaining Priority
- (iii) Parallel-Priority Interrupt

26.2.1 Polling

A polling procedure is used to identify the highest-priority source by software means.

26.2.2 Daisy-Chaining Priority

It is used to identify the highest-priority source by hardware means. It consists of a serial connection of all devices that request an interrupt.

26.2.3 Parallel Priority Interrupt

This method uses a register whose bits are set separately by the interrupt signal from each device. Priority is established according to the position of the bits in the register.

26.3 Maskable Interrupt

A hardware interrupt that can be ignored by setting a bit in an interrupt mask register's bit mask.

26.4 Non-maskable Interrupt

It is a hardware interrupt that lacks an associated bit-mask, so that it can never be ignored. It is often used for timers.

26.5 Inter Processor Interrupt

It is generated by one processor to interrupt another processor in a multiprocessor system.

26.6 Spurious Interrupt

It is a hardware interrupt that is unwanted.

Chapter 27

Modes of Data Transfer

Data transfer between the CPU and I/O devices may be handled in a variety of modes. Some modes use the CPU as an intermediate path, others transfer the data directly to and from the memory unit.

Data transfer to and from peripherals may be handled in one of the three possible modes:

- (i) Programmed I/O
- (ii) Interrupt-initiated I/O
- (iii) Direct memory access (DMA)

27.1 Programmed I/O

In programmed I/O, each data item transfer is initiated by an instruction in the program.

Once a data transfer is initiated, the CPU is required to monitor the interface to see when a transfer can again be made.

In the programmed I/O method, the CPU stays in a program loop until the I/O unit indicates that it is ready for data transfer. Usually, the transfer is to and from a CPU register and peripheral. Other instructions are needed to transfer the data to and from CPU and memory.

Here, the I/O device does not have direct access to memory.

27.2 Interrupt-Initiated I/O

An alternative to the CPU constantly monitoring the flag is to let the interface inform the computer when it is ready to transfer data. This mode of transfer uses the interrupt facility. In the meantime, the CPU can proceed to execute another program. The interface meanwhile keeps monitoring the device.

When the interface determines that the device is ready for data transfer, it generates an interrupt request to the computer. Upon detecting the external interrupt signal, the CPU momentarily stops the task it is processing, branches to a service program to process the I/O transfer, and then returns to the task it was originally performing.

27.3 Direct Memory Access (DMA)

In DMA, the interface transfers data into and out of the memory unit through the memory bus.

The CPU initiates the transfer by supplying the interface with the starting address and the number of words needed to be transferred and then proceeds to execute other tasks.

DMA transfer is also referred to as block mode.

A simple DMA Controller is a standard component in PCs.

Chapter 28

Input-Output Processor (IOP)

28.1 Overview

An IOP is a processor with direct memory access capability that communicates with I/O devices.

The IOP is similar to a CPU except that it is designed to handle the details of I/O processing.

Unlike the DMA controller that must be set up entirely by the CPU, the IOP can fetch and execute its own instructions. In addition, the IOP can perform other processing tasks, such as arithmetic, logic, branching, and code translation.

The memory unit occupies a central position and can communicate with each processor by means of direct memory access.

Chapter 29

Direct Memory Access

29.1 Overview

During DMA transfer, the CPU is idle and has no control of the memory buses. A DMA controller takes over the buses to manage the transfer directly between the I/O device and memory.

29.1.1 Bus Request

The bus request (BR) input is used by the DMA controller to request the CPU to relinquish control of buses.

29.1.2 Bus Grant

The CPU activates the bus grant (BG) output to inform the external DMA that the buses are in high-impedance state.

29.1.3 Burst Transfer

A block sequence consisting of a number of memory words is transferred in a continuous burst while the DMA controller is the master of the memory buses.

29.1.4 Cycle Stealing

It allows the DMA controller to transfer one data word at a time, after which it must return control of the buses to the CPU.

Chapter 30

Blocking and Nonblocking I/O

30.1 Blocking I/O

When an application issues a blocking system call, the execution of the application is suspended. The application is moved from the running queue to the waiting queue. After the system call completes, the application is moved back to the running queue.

A nonblocking call does not halt the execution of the application for an extended time. Instead, it returns quickly, with a return value that indicates how many bytes were transferred.

Chapter 31

Asynchronous I/O

An alternative to a nonblocking system call is an asynchronous system call.

An asynchronous call returns immediately, without waiting for the I/O to complete. The application continues to execute its code.

The completion of the I/O at some future time is communicated to the application through interrupt.

Chapter 32

Buffering

32.1 Overview

A buffer is a memory area that stores data while they are transferred between two devices or between a device and an application.

32.2 Spooling

A spool is a buffer that holds output for a device, such as a printer, that can not accept interleaved data streams.

32.3 Expansion Bus and PCI Bus

An expansion bus connects the internal hardware of the computer system (including the CPU and RAM) and peripheral devices.

The PCI Bus connects fast devices to processors.

Chapter 33

Polling

Polling is also referred to as busy-waiting. In this situation, when an I/O operation is required, the computer does nothing other than check the status of the I/O device until it is ready, at which point device is accessed.

It is sometimes also referred to as software-driven I/O.

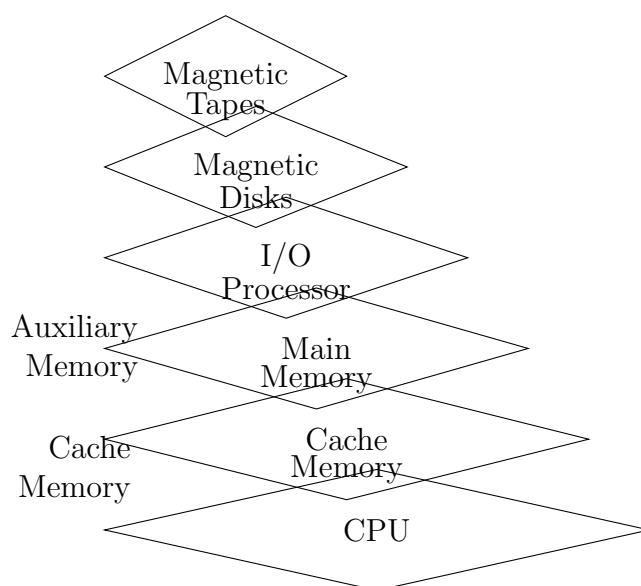
Chapter 34

Memory Management

34.1 Raster Display

The scanning process sweeps the beam from left to right across the screen n times from top to bottom. The beam is moved to the start of the next line (horizontal retrace) until position (vertical retrace).

34.2 Memory Hierarchy in a Computer System



The main memory occupies a central position by being able to communicate directly with the CPU and with auxiliary memory devices through an I/O processor.

The typical access time ratio between cache and auxiliary memory is about 1 to 7. Auxiliary memory average access time is usually 1000 times that of main memory.

34.3 Protection of Memory Space

We need to make sure that each process has a separate memory space. To do this, we need to be able to determine the range of legal addresses that the process may access and to ensure that the process can access only these legal addresses.

We can provide this protection by using two registers:

- (i) Base Register holds the smallest legal physical memory address
- (ii) Limit Register specifies the size of the range

Chapter 35

Static RAM

35.1 Overview

Static RAM consists of internal flip flops that store the binary information. The stored information remains valid as long as power is applied to the unit.

The Static RAM is easier to use and has shorter read and write cycles.

Chapter 36

Dynamic RAM

36.1 Overview

Dynamic RAM stores the binary information in the form of electric charge that is applied to capacitors. The capacitors are provided inside the chip by MOS transistors.

The stored charge on the capacitors tends to discharge with time and the capacitors must be periodically recharged by refreshing the dynamic memory.

The Dynamic RAM offers reduced power consumption and large storage capacity in a single memory chip.

Chapter 37

RAM and ROM Chips

37.1 RAM Chip

37.1.1 Bi-directional Bus

CS	RD	WR	Memory Function
*	*	*	Inhibit
0	1	*	Inhibit
0	*	1	Inhibit
1	0	1	Write
1	1	0	Read
1	1	*	Inhibit

37.2 ROM Chip

37.2.1 Uni-directional Bus

A ROM chip is organized externally in a similar manner; however, since a ROM can only read, the data bus can only be in an output mode.

For the same-size chip, it is possible to have more bits of ROM than of RAM, because the internal binary cells in ROM occupy less space than in RAM.

37.3 Memory Address Map

The addressing of memory can be established by means of a table that specifies the memory address assigned to each chip. This table is called a memory address map.

Component	Address Bus									
	10	9	8	7	6	5	4	3	2	1
RAM 1	0	0	0	*	*	*	*	*	*	*
RAM 2	0	0	1	*	*	*	*	*	*	*
RAM 3	0	1	0	*	*	*	*	*	*	*
RAM 4	0	1	1	*	*	*	*	*	*	*
ROM	1	*	*	*	*	*	*	*	*	*

Chapter 38

ROM

ROM is used for storing programs that are permanently resident in the computer and for tables of constants that do not change in value once the production of the computer is completed.

ROM is also Random Access Memory.

38.1 Bootstrap Loader

The bootstrap loader is a program whose function is to start the computer software operating when power is turned on. The ROM portion of main memory is used for storing an operating system (software) programs loads a portion of the operating system from disk to main memory and control is then transferred to the operating system, which prepares the computer for general use.

Chapter 39

Associative Memory

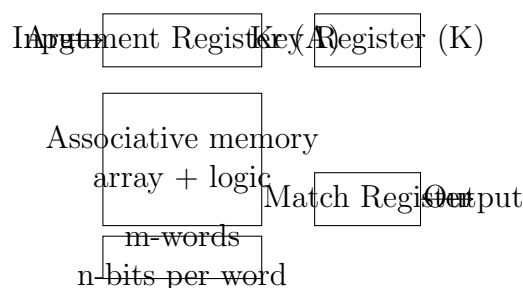
39.1 Overview

The time required to find an item stored in memory can be reduced considerably if stored data can be identified for access by the content of the data itself rather than by an address.

A memory unit accessed by content is called an associative memory or content-addressable memory.

An associative memory is more expensive than a random access memory because each cell must have storage capability as well as logic circuits for matching its content with an external argument.

39.2 Block Diagram of Associative Memory



The argument register A and key register K each have n bits, one for each bit of a word. The match register M has m bits, one for each memory word.

Each word in memory is compared in parallel with the content of the argument register. The words that match the bits of the argument register set a corresponding bit in the match register. The bits of the argument register set a corresponding bit in the match register.

Chapter 40

Hit Ratio

40.1 Overview

When the CPU refers to memory and finds the word in cache, it is said to produce a hit. If the word is not found in cache, it is in main memory and it counts as a miss.

The ratio of the number of hits divided by the total CPU references to memory (Hits plus miss) is the hit ratio.

For example: A computer with cache access time of 100ns, a main memory access time of 1000ns, and a hit ratio of 0.9 produces an average access time of:

$$0.9 \times 100 + 0.1 \times (100 + 1000) = 200 \text{ ns}$$

Chapter 41

Cache Memory

41.1 Locality of Reference

The references to memory at any given interval of time tend to be confined within a few localized area in memory. This phenomenon is known as the property of locality of reference.

If the active portions of the program and data are placed in a fast small memory, the average memory access time can be reduced, thus reducing the total execution time of the program. Such a fast small memory is referred to as a cache memory.

41.2 Mapping

The transformation of data from main memory to cache memory is referred to as a mapping process.

Three types of mapping procedures are of practical interest:

- (i) Associative Mapping
- (ii) Direct Mapping
- (iii) Set-associative Mapping

Chapter 42

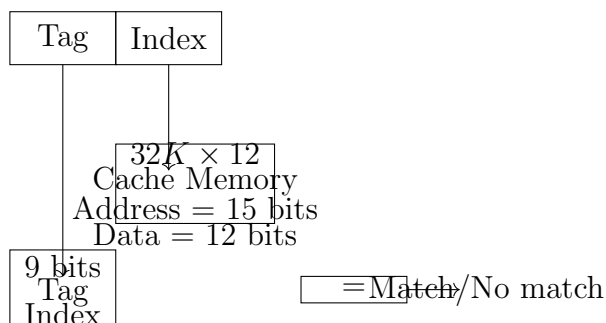
Direct Mapping

42.1 Overview

In the general case, there are 2^n words in cache memory and 2^n words in main memory address is divided into two fields:

- (i) k bits for the index field
- (ii) $(n - k)$ bits for the tag field

The direct mapping cache organization uses the n -bit address to access the main memory and the k -bit index to access the cache.



Each word in cache consists of the data word and its associated tag. When a new word is first brought into the cache, the tag bits are stored alongside the data bits.

When the CPU generates a memory request, the index field is used for the address to access the cache. The tag field of the CPU address is compared with the tag in the word read from the cache. If the two tags match, there is a hit. If there is no match, there is a miss.

Chapter 43

Set-Associative Mapping

43.1 Overview

The set-associative mapping is an improvement over the direct mapping in that each word of cache can store two or more words of memory under the same index address.

Each data word is stored together with its tag and the number of tag-data items in one word of cache is said to form a set.

Chapter 44

Writing into Cache

44.1 Overview

There are two ways:

- (i) **Write-through:** With every memory write operation updates main memory, with cache memory being updated in parallel if it contains the word at the specified address.
- (ii) **Write-back:** In this method, only the cache location is updated during a write operation. The location is then marked by a flag so that later when the word is removed from the cache, it is copied into main memory.

44.2 Cache Initialization

Each word in cache includes a valid bit to indicate whether or not the word contains valid data.

The cache is initialized by clearing all valid bits to 0. The valid bit of a particular cache word is set to 1 the first time this word is loaded from main memory and stays set unless the cache has to be initialized again.

44.3 Cache Coherence

In a shared memory multiprocessor system, with multiple processors having separate caches at its memory to keep the caches coherent by ensuring that any shared operand that is changed in any cache must be changed throughout the entire system. This is the cache coherence problem.

Chapter 45

Memory Binding

45.1 Overview

The binding of instructions and data to memory addresses can be done at any step along the way:

- (i) **Compile time:** If we know at compile time where the process will reside in memory, then absolute code can be generated.
- (ii) **Load time:** In this case, compiler generates relocatable code and final binding is delayed until load time.
- (iii) **Execution time:** If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time.

45.2 Dynamic Loading

With dynamic loading, a routine is not loaded until it is called. All routines are kept on disk in a relocatable load format.

Dynamic loading does not require special support from the operating system. It is the responsibility of the users to design their programs to take advantage of such a method.

Chapter 46

Dynamic Linking

46.1 Overview

Some operating systems support only static linking in which system libraries are treated like any other object module and are combined by the loader into the binary program image.

The concept of dynamic linking is similar to that of dynamic loading. Here, though, linking, rather than loading, is postponed until execution time. This feature is usually used with system libraries such as language subroutine libraries.

Under this scheme, all processes that use a language library execute only one copy of the library code.

Chapter 47

Virtual Memory

Virtual memory is a concept that permits the user to construct programs as though a large memory space were available, equal to the totality of auxiliary memory.

Chapter 48

Logical Versus Physical Address Space

48.1 Overview

An address generated by the CPU is commonly referred to as a logical address or virtual address, and the set of such addresses is referred to as address space or virtual address space or logical address space.

An address in main memory is referred to as a physical address, and the set of such addresses is referred to as physical address space or memory space.

The compile-time and load-time address-binding methods generate identical logical and physical addresses.

The run-time mapping from virtual to physical addresses is done by a hardware device called the memory-management unit (MMU).

Chapter 49

Contiguous Memory Allocation

49.1 Overview

For a memory request of size n , there are three methods are used to satisfy the request from a list of free holes:

- (i) **First Fit:** Allocate the first hole that is big enough
- (ii) **Best Fit:** Allocate the smallest hole that is big enough
- (iii) **Worst Fit:** Allocate the largest hole

Both the first-fit and best-fit strategies for memory allocation suffer from external fragmentation.

49.2 External Fragmentation

External fragmentation exists when there is enough total memory space to satisfy a request, but the available spaces are not contiguous; storage is fragmented into a large number of small holes.

One solution to the problem of external fragmentation is compaction. Compaction is possible only if relocation is dynamic and is done at execution time.

Another possible solution to the external fragmentation problem is to permit the logical address space of the processes to be non-contiguous, thus allowing a process to be allocated physical memory wherever the latter is available. This is accomplished through paging.

Chapter 50

Paging

50.1 Overview

Paging is a memory-management scheme that permits the physical address space of a process to be noncontiguous.

The basic method for implementing paging involves breaking physical memory into blocks of fixed size called frames or page-frames or blocks, and breaking logical memory into blocks of the same size called pages.

50.2 Mapping from Address Space to Memory Space

50.2.1 Page (p) Line Number (d)

1	0	1	0	0	1	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---

Table Address

Register

000

Page Table

P ₁	P ₂
----------------	----------------

Logical address

p	d
---	---

A presence bit in each location indicates whether the page has been transferred from auxiliary memory into main memory.

A 0 indicates the presence bit indicates that the page is not available in main memory.

Chapter 51

Translation Look-aside Buffer (TLB)

In paging, two memory accesses are needed to access some data: one for the page table entry and after for the data. This memory access is slowed by a factor of 2.

The solution to this problem is to use a special small fast-lookup hardware cache called a translation look-aside buffer. The TLB is associative, rapid memory.

The TLB contains only a few of the page table entries. When a logical address is generated by the CPU, its page number is presented to the TLB.

If the page number is found, its frame is immediately available and is used to access memory.

If the page number is not in the TLB (known as TLB miss), a memory reference to the page table must be made. In addition, we add the page number and frame number to the TLB so that they will be found quickly on the next reference.

Chapter 52

Valid-Invalid Bit

This bit is also kept in page table.

When this bit is set to valid, the associated page is in the process logical address space and is thus a legal page.

When the bit is set to invalid, the page is not in the process logical address space and is thus a legal page.

52.1 Demand Paging

With demand-paged virtual memory, pages are only loaded when they are demanded during program execution; pages that are never accessed are thus never loaded into physical memory.

Chapter 53

Page Replacement Algorithms

53.1 FIFO Page Replacement

A FIFO replacement algorithm associates with each page the time when that page was brought into memory. When a page must be replaced, the oldest page is chosen.

FIFO page replacement algorithm suffers from Belady's anomaly.

53.1.1 Belady's Anomaly

For some page replacement algorithms, the page-fault rate may increase as the number of allocated frames increases.

53.2 Optimal Page Replacement

Replace the page that will not be used for the longest period of time.

An optimal page replacement algorithm has the lowest page-fault rate of all algorithms and will never suffer from Belady's anomaly.

53.3 LRU Page Replacement

LRU replacement associates with each page the time of that page's last use. When a page must be replaced, LRU chooses the page that has not been used for the longest period of time.

This algorithm does not suffer from Belady's Anomaly.

Chapter 54

Thrashing

A process is thrashing if it is spending more time paging than executing.

Chapter 55

Local vs. Global Page Replacement

When a process incurs a page fault, a local page replacement algorithm selects for replacement some page that belongs to that same process.

A global replacement algorithm is free to select any page in memory.

Chapter 56

Byte Addressing vs. Word Addressing

56.1 Byte Addressing

In byte addressing, (1 word = 4 bytes):

0	1 word
1	
2	
3	

Address

0
1
2
3
⋮

Address

56.2 Word Addressing

In the above addressing scheme, the first word starts at address 0, and the first word are located at address 0.

The second word starts at address 4, and all bytes of the second word are located in address 4.

In the above addressing, in the first word all bytes of start at address 0 and all bytes of the second word are located in address a .